Errors

שורה 43: ערך התכונה "מ" חייב להיות tools.md, שגיאה-1 – שונות/תרגומי-טקסטים-קדומים מוקף במרכאות - "..." או '...' או ’...’ או ”...”

Tools

> Part of the guidelines for translating ancient texts (translation-guidelines.md), split out of it on 2026-10-21 to keep the core file lean. Section number §10, as in the guidelines.

10. Tools (`tools/`)

Script	What	Usage
`ebl_fetch.py`	Downloads an eBL corpus chapter and builds `reading-edition.md`, `glossary.md`, `manuscripts.md`, `bibliography.md`, `chapter.atf`. Caches in `_raw/`.	`python3 -I tools/ebl_fetch.py --list` · `python3 -I tools/ebl_fetch.py L 1 4 "Standard Babylonian" XI sources/eBL/Gilgamesh-SB-XI`
`etcsl2md.py`	ETCSL TEI XML → Markdown (Unicode transliteration, subscript indices).	`python3 -I tools/etcsl2md.py <c-or-t.xml> <out.md> [url]`
`cdl2md.py`	ORACC CDL JSON (from a project zip) → Markdown with lemmatization.	`python3 -I tools/cdl2md.py <zip> <member> <out.md> <title> <url>`
`dfhg2md.py`	DFHG API JSON (Greek fragments) → Markdown.	`python3 -I tools/dfhg2md.py <in.json> <out.md> <title> <api-url> [frags]`

More tools (2026-10-16):

- **`check_bible_quotes.py`**: checks every backticked, vocalized Bible quotation that is followed by a reference against `תנך-מנוקד/`. It normalizes Unicode, allows prefixes and quotations that run on into the next verses, and defaults to Genesis. `python3 -I tools/check_bible_quotes.py <files...>`; `--show א כח "בראשית ח"` prints a verse. **Run it before finishing any project.** Note: the Psalms and Proverbs files use a hyphen-minus for maqaf, so keep the `-` in quotations from them.
- **`ebl_fetch.py`** (fixed 2026-10-16): notes are now taken from the JSON (the ATF omits many), and range lines (`66-67`) are labeled.

More tools (2026-10-16, creation corpus):

- **`ebl_fetch.py`** (fixed again): a line with several **variant reconstructions** now prints all of them (**`N (variant, other manuscripts):`**) with their manuscripts; before, the second variant and its manuscripts were silently dropped. Re-run it on older eBL folders if a line looks incomplete.
- **Project-level verification pattern** (`סיפורי-הבריאה-המסופוטמיים/1-אנומה-אליש/verify.py`): checks the annotated entries against eBL's line list, the translation lines against the annotated `>` lines, every pair of eBL "parallel lines" (bracket-only differences listed separately), and a fixed passage against its table in `renderings.md`. Copy and adapt it for any eBL-based multi-tablet project. The same folder has `build_translation.py` (one reading file from per-tablet files) and `build_parallels_index.py` (merges per-tablet parallels tables into the corpus `parallels-index.md`, ordered by verse).

More tools (2026-10-16, creation corpus project 2):

- **`seal2md.py`**: a saved SEAL text page → Markdown (catalogue fields; the text table line by line, with bilingual continuation lines marked `↳`; SEAL's per-word lemmas; the translation). `python3 -I tools/seal2md.py <node-N.html> <out.md> <url>`. SEAL returns an error page to curl's default User-Agent: download with a browser UA (see the docstring).
- **`cdl2md.py --morph`**: the ORACC converter now prints each word's ePSD2 `morph` and base (`{ba.n.da:~;a,bi.a;nud}`), the Sumerian "check the stem" (§4.6).
- **`check_bible_quotes.py`** (fixed 2026-10-17): the loose book key dropped every ' and ו, so `יהושע` resolved to `הושע`; it now keeps the first letter and tries the exact name first.
- **eBL fragmentarium transliteration search**: `GET /api/fragments/query?transliteration=<signs>` finds every fragment with a sign sequence (e.g. `ul-le-gar-ra` → VAT 9307 = KAR 4). Use it to look for duplicates of a text before declaring it single-manuscript.
- **Project-level `verify.py` for a multi-text project** (`סיפורי-הבריאה-המסופוטמיים/2-טקסטי-בריאה-ועדן/verify.py`): per text, checks the annotated entries against the edition's line list (eBL reading edition or ETCSL transliteration) and the translation lines against the annotated `>` lines; `build_translation.py` there joins per-text files.

More tools (2026-10-17, law and treaty corpus):

- **`cdl2md.py --norm`**: adds a `norm:` line (the project's normalization of each word; SAAo, RINAP etc.) and prints the contextual sense after the guide word (`[(treaty-)oath=treaty]`).
- **`oracc_html2tr.py`**: a saved ORACC per-text HTML view → Markdown translation (units labelled with the transliteration lines they span, section headings as `##`). `python3 -I tools/oracc_html2tr.py <in.html> <out.md> <title> <url>`. Use it for any ORACC project: the zips have no translations.
- **Project tools in `חוקים-ובריתות/1-אמנת-הירושה-של-אסרחדון/`**: `build_data.py` + `make_parts.py` (align a composite with a later copy by the SAAo labels and cut per-part data files for the translators), `verify.py` (entries, one `>` per entry, **reserved words**, fixed formulas and key-lemma roots, identical normalizations), `build_translation.py` (clean translation with § headings and footnotes from a `הערת שוליים` field), `build_parallels_index.py` (corpus index, Deuteronomy grouped by chapter).

- **sources_sync.py** (2026-10-21): re-obtains the git-ignored sources. `status` / `fetch` / `verify` (work tier by default; `--all` adds the archive tier of old scans and photos; `--only REGEX`, `--dest DIR`, `--jobs N`); `snapshot` rewrites `sources/MANIFEST.json` where the files are. The recipes are in `sources/fetch-recipes.tsv`. It reports `CHANGED` when an online edition changed since our copy, and names the hosts blocked by a cloud allowlist. Text files are compared with their "generated/retrieved" dates blanked. Never extracts `.exe`/`.dll` from a zip. **`snapshot` in a partial checkout** (a cloud session without the archive tier, or with files that are `CHANGED`) drops the absent items and re-stamps the changed ones: there, add only the new items to the committed manifest (as the lead did on 2026-10-21). On a certificate error it retries with the public intermediates in `tools/certs/` (ORACC omits its intermediate; 2026-10-21), then falls back to curl with the same certificates.

When you write a tool that another session could reuse, put it here with a usage docstring and add a row to this table.

More tools (2026-10-17, the Laws of Hammurabi):

- **cdli_composite2md.py**: a CDLI composite ATF with `#tr.ts` / `#tr.en` lines → Markdown, optionally aligned line by line with one witness ATF (stele/tablet) by the transliteration sequence, and a JSON-lines file. Get the ATF from `https://cdli.earth/artifacts/<id>/json` (`[0]['inscription']['atf']`; witnesses' ATF inside `[0]['witnesses']`). `python3 -I tools/cdli_composite2md.py <composite.atf> <out.md> <title> <url> [--witness <w.atf> <siglum>] [--jsonl <out>]`.
- **Project tools in `חוקים-ובריתות/2-חוקי-חמורבי/`**: `lh_config.py` (files, topics, composite parser, stele alignment, SEAL mapping), `build_data.py` (per-file translator data: composite + stele + SEAL lemmas + rank-3), `verify.py` (entries per file with placeholders, § sections against the composite, reserved words, formula patterns, identical normalizations, translation = `>` lines; `--strict`), `build_translation.py` (topic and § headings, footnotes, placeholders). **Corpus builder** `חוקים-ובריתות/build_parallels_index.py` merges every project's Part tables (Exodus and Deuteronomy by chapter); it replaces the project-1 builder.
- **check_bible_quotes.py** (fixed 2026-10-17): references to `דברי הימים א/ב` were skipped; now matched.

שורה 43: ערך התכונה "מ" חייב להיות שונות/תרגומי-טקסטים-קדומים/שגיאה-1 – tools.md, מוקף במרכאות - "..." או '...' או ’...’ או "..."

More tools (2026-10-18, the other law collections, in `חוקים-ובריתות/`): `laws_config.py` (projects LE, MAL, MALB, LI, UN: CDLI composite parser with exemplar links, ORACC MAL parser with the § units of the translation, files, TOPICS; Hebrew helpers), `laws_build_data.py KEY <dir>` (translator data per file), `laws_verify.py KEY [n...] [--strict]` (entries, § ranges, reserved and lemma-gated words, LH's fixed formulas for Akkadian and their equivalents for Sumerian, identical normalizations inside and across collections, translation = `>` lines), `laws_build_translation.py KEY`

(topic headings by § or by composite label `@label`), `laws_lookup.py '<regex>' [--heb] [--proj …]` (how LH and the new projects rendered a word); `build_parallels_index.py` now reads all seven projects. The common translator brief: `חוקים-ובריתות/translator-brief--laws.md`.

More tools (2026-10-19, creation corpus projects 3–4):

- **`sumerian_workbench.py`**: the Sumerian workbench that project 2 asked for. `lemmas <c.xml> FROM-TO` (ETCSL's TEI lemmas per word), `line <c.xml> FROM-TO` (the same, plus for every verb the ORACC morph of the same form wherever it is parsed in epsd2-literary, or "no parsed instance: our analysis"), `morph '<form>'`, `conc '<regex>' [--lemma]` (ETCSL-wide concordance, for formulas), `gloss '<regex>'` (ePSD2 glossary). The ORACC form index is built once into `sources/ORACC/_bulk/epsd2-literary--form-index.json` (gitignored). Run with `python3 -I`.
- **`סיפורי-הבריאה-המסופוטמיים/sumerian_tools.py`**: `verify KEY [--strict]` (coverage against the ETCSL line list over several annotated files, one `>` per entry, reserved strings licensed by the Sumerian word, name traps, identical transliterations with different Hebrew), `build KEY` (the clean translation generated from `translation--intro.rtl.md` + the annotated `##` headings and `>` lines, with `הערת שוליים` bullets as footnotes), `grep all '<regex>'` (projects 2–7). Add a project as a `PROJECTS` entry. With `translator-brief--sumerian.md` (the reusable brief). `אנומה--1 אליש/build_parallels_index.py` reads projects 3–7 (sigla B&F, EWO, FI, H&P, S&W). **Since 2026-10-22 (Round 3 run 1):** projects 5–7 (`gates57`) add `RESERVED_5_7` (substring gates) and `TOKEN_GATES_5_7` (whole-word gates: each Hebrew token is tried with every prefix split – ,ו ה, ב, ל, מ, כ, ש and pairs – and needs the licensing Sumerian word; `TOKEN_EXEMPT_5_7` lists ordinary words a split would turn into a gated one, e.g. `מנחה`, `מעצב`, `לחם`, `בקר`); the Sumer / Urim name traps; the עבד gate also licensed by *arad*; brackets are stripped from both the Hebrew and the transliteration before every gate (a half-bracketed word, `ק[יץ]`, is read whole). Run brief: `translator-brief--sumerian--5-7.md` with `…--shared-lemmas.txt`.

More tools (2026-10-19, the wisdom corpus, in `ספרות-החכמה-והצדיק-הסובל/`): `wisdom_config.py` (files with eBL line ranges, so one eBL chapter can be split over several annotated files; eBL readers; the Gilgameš corpus's Hebrew for cross-corpus parallels) and `wisdom_tools.py – pars KEY` (eBL parallels incl. cross-text `L I.4` links to Gilgameš, and identical normalizations, with the Hebrew already given), `verify [KEY…] [--strict]` (entries, translation = `>` lines, reserved strings licensed by an eBL lemma *or a normalization token*, vocalization traps gated by lemma, repeated lines), `build` (per-file and per-project clean translations generated from `translation--intro-<KEY>.rtl.md` + the annotated file; never edited by hand), `grep '<regex>' [--gilg]`, `lemma '<regex>'` (how a lemma was rendered so far: the cheapest consistency tool); `build_parallels_index.py` (Job first, then Psalms, Qohelet, Proverbs, Lamentations, Isaiah, Jeremiah, the Torah). `translator-brief.md` there is the reusable brief for Akkadian eBL texts.

More tools (2026-10-19, the city-lament corpus, in `קינות-על-ערים/`): `lament_tools.py` – `verify KEY [--strict]` (entries against the ETCSL line list **with segment labels** `A1…` for fragmentary segments, the backticked transliteration equal to ETCSL's line, reserved and gated strings licensed by a Sumerian word, name traps, **fixed refrains checked by regex (FORMULAS)**, translation = `>` lines, identical lines with different Hebrew), `build KEY`, `grep KEY|all` (this corpus + the creation corpus's Sumerian texts), `lines KEY FROM-TO` (ETCSL's lines for a brief); `build_parallels_index.py` (Lamentations first, then Psalms, Jeremiah, Ezekiel, Isaiah, the other Prophets, the Torah, Daniel); `translator-brief--laments.md` (the reusable brief for Sumerian liturgical texts). Add a project as a `PROJECTS` entry.

More tools (2026-10-20, the temple-building corpus, in `בניין-המקדש/`): `temple_tools.py` – `conc GU [<out>]` (the label concordance: ETCSL composite numbers ↔ RIME column-line labels ↔ ORACC, with a per-line similarity check), `data GU FROM TO <out>` (a translator's line-data file: ETCSL transliteration, ORACC lemma + morph, and the two rank-3 translations – ETCSL's paragraphs mapped from composite numbers to labels, ETCSRI's units – at their first line), `verify GU [--strict]` (as `lament_tools.py`, for labels `A.ix.10`; the reserved Tabernacle words and the gated building words), `build GU`, `grep GU|all`, `lines`; `build_parallels_index.py` (Exodus first, then Kings, Chronicles, Ezekiel, Genesis …); `translator-brief--temple.md` (the reusable brief for Sumerian royal and building texts). ETCSRI per-text HTML + `oracc_html2tr.py` gives the ETCSRI translation of any royal inscription.

More tools (2026-10-21, the netherworld corpus, in `העולם-התחתון/`): `netherworld_tools.py` – `verify IS|IN [--strict]` (entries vs. the eBL labels incl. `NA1…` and the ETCSL labels; the backticked source = the base text; reserved and gated Hebrew licensed by the eBL normalization/lemma or the Sumerian word; name traps; fixed formulas, also across the two languages; translation = `>` lines; identical sources with different Hebrew), `build IS|IN`, `lines IS|IN FROM TO` (base lines with eBL lemmas / ORACC lemma + morph), `grep all '<regex>'` (the `>` lines of every corpus, niqqud stripped); `build_parallels_index.py` – **generates** the per-project parallels tables from the annotated **הקבלה** bullets and merges them by Bible verse into `parallels-index.md` (the key findings kept): no hand-copied tables, so the index cannot drift from the annotations; recommended for new corpora.

More tools (2026-10-22, the netherworld corpus, Round 3 run 5): `netherworld_tools.py` now has keys **NE** (Nergal and Ereškigal: two eBL chapters, the MB labelled `MB.<label>`, then the SB) and **GEN** (Bilgameš, Enkidu and the Netherworld: an ETCSL composite with **several recensions** – `versions=[(heading, prefix)]` gives each bold version heading of `transliteration.md` its own label prefix, `U58.`, `U59.A…`, `MT.B…`; ORACC aligned by position, now also with labels like `Ver.1, 12`); **xii FROM TO** prints each Sumerian line with its Gilgameš XII counterparts and their Hebrew (`XII_CONC`, aligned by content and corrected by the translators: George's "BN" numbers run ~1 ahead of ETCSL) – the pattern for any Sumerian text with an ancient Akkadian translation already translated; new reserved / gated words (`גן`; `רקיע`, בדל, `לילית`, `מלאך`, `סולם` by *kiri*$_6$, `היכל`, `פוכו`),

traps (Anu `אָנוּ`, Kakka `כַּכָּה`, Erra `אֶרָה`, *pukku*, *mekkû*, *eṭemmu* `אֵט`, Sumer `שׁוּמֶר`) and formulas for both texts. Fixed: variant parentheses `(1 ms. has instead: …)` are removed before the italic notes (their words leaked into the formula check); `⌜ ⌝` are removed before the formula check; the gate licences read eBL's *h* as *ḫ*; a heading with a single label (`## b+50: …`) is stripped in `build`. `build_parallels_index.py` reads projects 3–4 and skips missing files.

More tools (2026-10-21, the legends-of-kings corpus, in `אגדות-מלכים/`; run with `python3 -I -B`): `kings_tools.py` – `verify EA|SB [--strict]` (entries vs. the base line list: ETCSL 1.8.2.3, King 1907 labels `I.n` / `II.n`; transliteration = base text; reserved and gated strings tested on the transliteration with and without brackets; name traps in lines and headings; fixed formulas; translation = `>` lines; identical transliterations with different Hebrew), `build EA|SB`, `grep EA|SB|all`, `lines`, `pars KEY FROM TO` (each line's earlier copies, ratio ≥ 0.75, with their Hebrew: for command/delivery texts), `conc EA` (ETCSL ↔ ORACC Q000369, whose variant lines are unlabelled), `data EA FROM TO out` (a translator's data file with ETCSL, ORACC lemma + morph, ETCSL's English and Mittermayer's German from the OCR); `build_parallels_index.py` (Genesis first, then Exodus).

More tools (2026-10-22, Etana, `אגדות-מלכים/kings_tools.py` key **ET**): `verify ET [--strict]` (585 entries over six eBL chapters, labels `<chapter>.<eBL label>`; the reserved and gated words licensed by the normalization **and the eBL lemma names** – RESERVED_ET; name traps for Etana, Šamaš / the sun, Anu, Sîn, Šakkan, *eṭemmu*, *bēru*; fourteen Etana formulas), `build ET` (the eBL label only, under each version's `#` heading taken from the `<!-- כותרת־נוסח: … -->` line of each annotated file), `data ET FROM TO out` (each line's eBL block + its **related lines in all six versions** – eBL's `Parallels:` both ways and near-identical normalizations – with the Hebrew already given), `pars ET FROM TO`, `grep all` (now also the Akkadian corpora: Gilgameš, flood, Enūma eliš, Adapa, KAR 4, wisdom, Ištar's Descent, love lyrics, laws). Reusable for any multi-version eBL text: a new PROJECTS entry with `base='ebl'` and its chapter list. `build_parallels_index.py` reads the Etana tables (rows `ET <chapter>.<label>`).

More tools (2026-10-21, the everyday-voices corpus, in `קולות-היום-יום/`): `kol_tools.py` – `units`; `data UNIT [FROM [TO]]` (a translator's line data: label, transliteration, normalization, lemmas + ORACC morph, the rank-3 translation; per proverb for the proverbs); **`fill FILE`** (replaces each `` `@` `` transliteration placeholder by the exact base line: drafting with `@` removed every retyping mismatch – recommended for every corpus); `verify [UNIT|PROJECT …] [--strict]` (coverage and order, transliteration = base, reserved and gated strings licensed by the line's transliteration + normalization + lemmas, anchored at word start, name traps, formulas, identical sources with different Hebrew, translation = `>` lines); `build PROJECT`; `grep (PROJECT|all|every) '<regex>'`; `lookup '<regex>'` (how the earlier corpora rendered a word); `build_parallels_index.py [--stats]` – generates `parallels-index.md` straight from the annotated **הקבלה** bullets (no per-project tables), grouped by book with the Song of Songs, Proverbs and Qohelet first, keeping the hand-written key findings.

- **`ebl_fetch.py`** (fixed 2026-10-22, Etana): a reconstruction line that opens with a logogram in another language (`25. %sb U₄.1.KAM@v.TA.AM₃ | %n inaṣṣarū …`) was not recognized (only `%n` was), so it was glued to the previous line as a "variant" and its own N was empty. Now any `%<language>` opening is a reconstruction line, and the language markers are dropped from N. Re-run the tool on an older eBL folder if a line has an empty N.
- **`ebl_fetch.py`** (2026-10-21, Round 3 run 5): prints every **non-English translation** too, as **`T (de):`** (eBL's Nergal and Ereškigal MB has only Mitto's German; before, the chapter looked unttranslated). Re-run it on an older eBL folder to add the German where eBL has it (no normalization changes).

More tools (2026-10-22, the wisdom corpus, Round 3 run 4, in `ספרות-החכמה-והצדיק-הסובל/`): `wisdom_config.py` now takes an **ETCSL composite** beside the eBL chapters (`ETCSL_KEYS`: the line list with segment labels, ETCSL's own lemmas `lemma[label]`, its English at each paragraph's first line; the Instructions of Šuruppag), the eBL category per file (`CAT`: L III for the hymns), `COPIES` (lines another corpus translated first – Šuruppag 1–13 from the flood corpus – checked identical) and `FILLED` (files whose source lines must equal the base); `wisdom_tools.py` – **`fill KEY`** (the `@` placeholders, as `kol_tools.py fill`), **`sux KEY`** (Sumerian lines identical or similar, ratio ≥ 0.75, to a line already translated in the proverbs or the flood corpus, with its Hebrew), gates for the Torah's laws of weights, bribes, interest, parents and long life (§11.6 of the renderings), per-file vocalization traps (`KEY_TRAPS`: Šamaš / the sun). Fixed: lemma licences now split two-lemma words and equate eBL's *h* with *ḫ*; eBL's chapter-less parallels (`L II.4 10`) are read. `build_parallels_index.py` reads projects 3–6.

More tools (2026-10-21/22, the prophecy-and-kingship corpus, in `נבואה-ומלכות/`; run with `python3 -I -B`): `prophecy_tools.py` – `units`; `data UNIT [FROM [TO]]` (SAAo transliteration + normalization + lemmas + the **rulings and breaks** of the tablet + Parpola's English; for an eBL text the normalization, the variant normalizations and every manuscript's ATF); `fill FILE [--refill]` (`--refill` re-copies lines already filled, after a base text was regenerated); `shared` (normalizations repeated across the units, with the Hebrew already drafted); `verify [UNIT|PROJECT …] [--strict]` (as `kol_tools.py`: coverage, base line, reserved and gated Hebrew licensed by the transliteration + normalization + lemmas, name traps, formulas by regex, identical normalizations with different Hebrew, translation = `>` lines); `build PROJECT`; `grep`; `lookup` (searches every backticked source line of the earlier corpora, transliteration or normalization). Units are ORACC SAAo texts (`kind: saa`) or eBL chapters (`kind: ebl`); an eBL line without a normalization takes its lemma forms. `build_parallels_index.py` (from `kol_tools`'s: generated from the **הקבלה** bullets; the Torah first; **the hand-written key findings must stand above the line `<!-- generated below: do not edit -->`, or a rebuild drops them**).

- **cdl2md.py** (fixed 2026-10-22): a logogram that writes two words (NA AMA—MAN = *ummi šarri*, DUMU—DUMU-ka) is one graphic word with two lemma nodes (cof-head / cof-tails); the converter printed it once per lemma. Fixed; files converted before that date may show such words twice in the transliteration (the normalization and lemmas were right). The SAA 9 files were regenerated. **Still affected (2026-10-22): EST** – sources/ORACC/saao-saa02--Q009186--Esarhaddon-Succession-Treaty/ (118 lines of Q009186, 41 of the Tayinat copy) and the transliterations copied into חוקים-ובריתות/1-אמנת-הירושה-של-אסרחדון/ (the Hebrew is not affected: it was made from the normalization). The other ORACC conversions show no doubled compound. Regenerate EST and re-copy its transliterations in a session on that corpus (its verify.py compares them).

More tools (2026-10-21/23, the witchcraft-and-incantations corpus, in כשפים-ולחשים/; run with python3 -I -B): maqlu_tools.py – fill FILE (the eBL normalization for every `@`), pars KEY (every line repeated anywhere in Maqlû, identical or ratio ≥ 0.8, and the fixed lines of renderings.md §9, with the Hebrew already given), **lem KEY FROM [TO]** (each word's lemma candidates from ORACC cmawro's Akkadian glossary – a lookup by normalized form, for eBL chapters without lemmas; the index is cached in sources/ORACC/_bulk/cmawro--norm-index.json), norm '<regex>' (how an Akkadian word was rendered so far), incipits (every incantation's first line with its Hebrew: for the Ritual Tablet and Tablet VIII), lines, verify [KEY...] [--strict] (entries vs eBL's labels, the backticked N = eBL's, translation = > lines, RESERVED gates licensed by the bracket-free N, TRAPS, FORMULAS (the opening קָסָם., the closing לַחַשׁ, קָסָם., the rubric נוּסַח לְהַתָּרַת רִיר, the refrains), the fixed lines of §9 (punctuation and niqqud ignored), identical normalizations with different Hebrew), build, grep '<regex>' [--all] (every corpus with --all); build_parallels_index.py (one project, one generated table per tablet; adapted from the netherworld corpus). **tools/ebl_fetch.py** (2026-10-21/23): a reconstruction line is recognized by its bare line label too (160b. KA.INIM.MA ..., 121a. [...] ..., B+16a), in addition to the language-marked lines fixed for Etana (25. %sb ...); Maqlû's incantation openings and rubrics had been glued to the line before. Checked identical on Ludlul V.

- **tools/harmonize_survey.py** (2026-10-23, the post-Round-3 harmonization): which Hebrew each corpus uses for a source word, across every annotated*.rtl.md of the folder. src REGEX [--heb REGEX] [--show N] (entries whose transliteration or parse names the source word, per project; with --heb, split by whether the > line has the Hebrew); heb REGEX [--src REGEX] (entries whose > line has the Hebrew, niqqud stripped; with --src, the collisions: the same Hebrew for another source word); words REGEX (the Hebrew words of the matching entries, counted); parse REGEX (the parse segments that name the word: note that their quoted glosses are dictionary meanings, not always the chosen Hebrew). Anchor Hebrew regexes at word start and end ((^|[^א-ת])[בלכמהש]?ו?(...)). Run with python3 -I.

More tools (2026-10-21, printed rows; moved 2026-10-24): **`layout/rows-tool.ts`** – measures, proposes and checks the printed rows of a `translation.rtl.md` (its `translation.rtl.layout.ts`, through `layout/translation-layout.ts` - the translations' layout logic, beside it), in the same Chromium, fonts and A4 width as the PDF printer. It never writes the module: `try` and `propose` print the row starts as constants to put into it. `bun layout/rows-tool.ts status | report <file> [--text] [--widths] [--run=N] [--review] | try <file> --run=N --move=a>b | propose <file> [--fix] | check <file> | resolve <file> | template <file> [--rules] | compare <file> --with=<module> | md5 <file> | preview <file> --pages=2-4,9 --out=<dir>`, and `--layout=<module>` / `--out=<path>`. `bun run layout-status` (in `_RTL-EDITOR`) lists every file whose layout is missing, obsolete (the file changed since), flagged or broken. Used by layout sessions; see `layout/layout-guidelines.md` §3.